

클라이언트 프로그래머 포트폴리오

한국공학대학교 임찬진

1. 졸업 작품 프로젝트

[프로젝트명] S.T.L (Slay The Lord)

<https://github.com/AACHANJINAA/PIP>

S.T.L (Slay The Lord)은 3D 다크 판타지 멀티플레이 액션 RPG(Soul-like)입니다.

본 프로젝트는 상용 게임 엔진(Unreal, Unity)을 사용하지 않고, 오직 DirectX 12 기반의 자체 C++ 렌더링 엔진과 IOCP 기반의 고성능 멀티플레이 서버를 직접 설계하고 구현해 낸 게임입니다.

오픈소스 물리엔진(Jolt Physics)을 서버에 연동하여, 타격 및 피격 판정과 이동 처리를 100% 서버에서 검증하는 안전하고 정교한 전투 동기화를 구현했습니다.

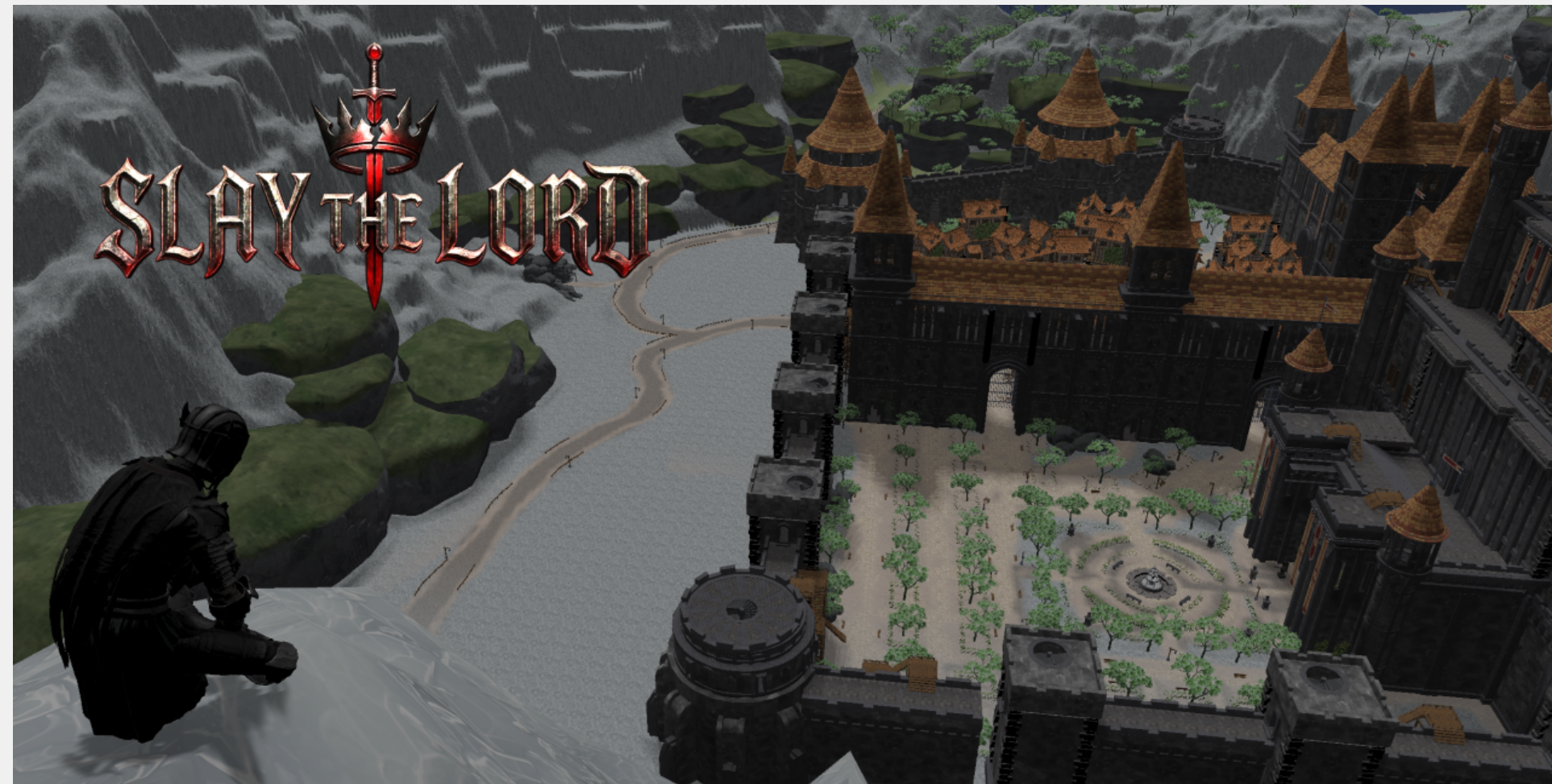
다크 판타지 특유의 차갑고 묵직한 분위기를 살리기 위해 물리 기반 렌더링(PBR) 및 이미지 기반 라이팅(IBL) 파이프라인을 자체 구축했습니다.

외부 라이브러리(Assimp 등)에 의존하지 않고, 언리얼에서 추출된 glTF 2.0 표준 데이터를 직접 파싱하는 모듈을 개발했습니다.

이를 통해 복잡한 스키닝 애니메이션과 3D 메쉬 데이터를 자체 엔진의 구조에 맞게 완벽히 변환하고 제어할 수 있는 기반 기술을 확보했습니다.

해당 게임 제작은 26년 7월초에 완료 예정입니다.

게임 제작 중에서 제가 담당한 glTF 2.0 표준 데이터의 material 부분 파싱과 PBR 및 IBL 파이프라인에 대해 설명하겠습니다.



첫 타이틀 썬의 모습

1. 졸업 작품 프로젝트에서의 역할

자체 D3D12 엔진의 그래픽스 코어 구현 및 엔진 간 데이터 파이프라인 자동화 담당

- 주요 역할. d3d12 및 hlsl을 활용한 그래픽스 렌더링 (PBR&IBL)

* PBR/IBL 파이프라인 구축:

Cook-Torrance BRDF 및 Split-Sum Approximation 기반의
사실적인 물리 기반 렌더링 설계 및 구현하였습니다.

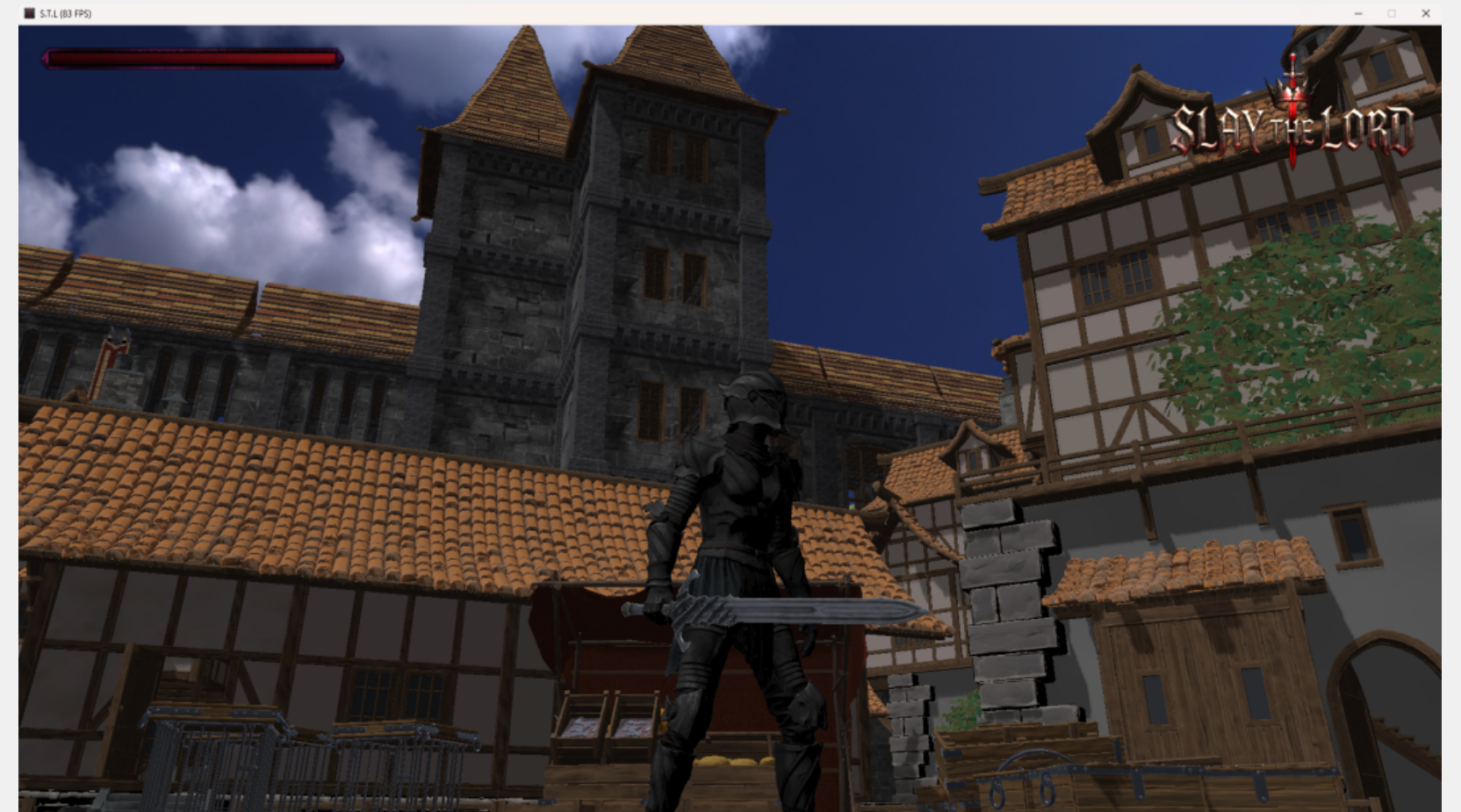
- 구현 역할 02. Asset 파이프라인 및 최적화 (Unreal to 자체 엔진)

* Unreal을 Asset Editor를 사용하였습니다.

* Foliage를 Unreal에서 export 및 자체 엔진 파싱 및 렌더링

Foliage와 그림자 렌더링 시 Draw Call 감소를 위한 Instancing 기술 적용

중복되는 메시는 하나만 추출하고 추가적으로 transform의 정보를 담은 JSON으로
추출하여, 자체 엔진에서 실시간 복원하는 로더 환경 자동화 및 Instancing 기술 적용
필요한 내용만 추출하기 위해 mesh는 지형 별로 뽑을 수 있게하고,
Foliage는 지형 및 이름 별로 추출할 수 있게 제작하였습니다.



인게임 모습 중 한 장면

1 - 1. 렌더링 시스템(PBR)

메모리 대역폭 최적화를 위한 텍스처 채널 패킹(Channel Packing) 처리

언리얼 엔진에서 추출한 glTF 2.0포맷의 텍스처에서 ORM (Occlusion, Roughness, Metallic) 맵을 읽어와 하나의 텍스처(채널별 분리)로 메모리를 최적화하여 셰이더에 전달하였습니다.

물리 기반 렌더링(PBR)을 구현하기 위해서는 Albedo 외에도 Roughness, Metallic, AO 등 여러 장의 텍스처 데이터가 필요합니다.

해당 텍스처들을 각각 개별 텍스처로 바인딩하면 GPU 메모리 낭비와 텍스처 샘플링 오버헤드가 발생합니다.
이를 해결하기 위해 glTF 표준 파이프라인에 맞춰 하나의 텍스처(ORM Map)의 RGB 채널에 데이터를 압축하였습니다.

```
// 2. ORM (Occlusion, Roughness, Metallic) 값 설정  
float3 orm = g_txORM.Sample(g_samLinear, In.TexCoord).rgb;
```

```
float roughness;  
float metallic;  
  
if (HasMetallicRoughnessTexture > 0)  
{  
    roughness = orm.g * RoughnessFactor;  
    metallic = orm.b * MetallicFactor;  
}  
else  
{  
    roughness = RoughnessFactor;  
    metallic = MetallicFactor;  
}
```

좌측 코드는 hlsl 셰이더 코드의 일부입니다.
픽셀 셰이더 단계에서 압축된 ORM 텍스처를 읽어와 G(Roughness), B(Metallic) 채널을 분리하고, glTF 2.0 메쉬의 기본 Material Factor 값과 곱하여 최종 값을 계산하는 과정입니다.

1 - 2. PBR

빛의 물리적 특성을 근사한 Cook-Torrance BRDF 방정식 구현

미세면 분포 (D_GGX):

표면의 미세한 거칠기(Roughness)에 따라 빛이 퍼지는 정도를 계산하여 사실적인 하이라이트(Specular)를 만듭니다.

기하 감쇠 (Vis_SmithJointApprox):

미세면끼리 그림자를 지게 하거나 빛을 가리는 현상을 계산합니다.

프레넬 반사 (F_Schlick):

빛을 바라보는 각도(시야각)에 따라 반사율이 달라지는 프레넬 효과를 Schlick의 근사식으로 구현했습니다.

기본 roughness를 둔 이유는 roughness 수치가 0인 텍스처가 들어왔을 때 Vis_SmithJointApprox 함수에서 a를 분모로써 나누기 하는데 NaN이 되어 해당 Mesh가 까맣게 되거나 하얗게 번쩍이는 등의 버그가 일어나서 입니다.

그리고 아까 PBR 설명할 때도 roughness의 기본 값을 주고 해당 함수에서도 준 이유 중에 또 하나로 현실 세계에서 모든 물체에는 미세한 굴곡이 존재합니다.
그래서 물리 기반 렌더링이란 의미인 PBR에 맞게 roughness = 0.0인 물체는 존재 하지 않으니 하한선을 주었습니다.

```
float NoL = saturate(dot(N, L));
float NoV = saturate(dot(N, V));
float NoH = saturate(dot(N, H));
float VoH = saturate(dot(V, H));

roughness = max(roughness, 0.15);
float a = roughness * roughness;

float D = D_GGX(a, NoH);
float Vis = Vis_SmithJointApprox(a, NoV, NoL);
float3 F = F_Schlick(SpecularColor, VoH);

// Diffuse
float3 kD = (1.0 - F) * (1.0 - metallic);
float3 diffuse = kD * albedo;

// Specular
float3 spec = D * Vis * F;

Lo += (diffuse + spec) * radiance * NoL;
```

좌측코드는 hlsl 셰이더 내 PBR 조명 연산의 핵심인 Cook-Torrance BRDF 연산 과정입니다.

픽셀 셰이더 단계에서 표면의 미세한 굴곡(D_GGX), 기하학적 그림자(Vis_Smith), 시야각에 따른 반사율(F_Schlick)을 계산하여 specular를 도출하고,

물리적인 에너지 보존 법칙(Energy Conservation)을 적용하여 금속성(Metallic)과 반사된 빛(F)의 양만큼 난반사(Diffuse) 비율을 감소시켜 최종 렌더링 방정식을 완성하는 과정입니다.

1 - 3. IBL

IBL 환경광의 난반사(Diffuse IBL)를 처리

기존 IBL 처리 시 기본 SkyBox, Diffuse CubeMap, Specular CubeMap, BRDF LUT를 사용합니다.
BRDF LUT는 무겁지 않은 텍스처라 상관 없지만, 각 4K 텍스처로 맵을 가지고 있는 텍스처들을 모두 처리하기에는 최적화가 어려웠습니다.

그래서 Diffuse CubeMap은 CubeMap 텍스처를 픽셀마다 샘플링하는 대신, 환경광을 9개의 구면 조화 함수(Spherical Harmonics, SH) 계수로 txt 파일에 적어둔 상수로 넘겨받은 뒤 수학적으로 복원하여, 메모리와 연산 비용을 최적화하였습니다.

```
// SH 계수를 이용한 Irradiance 계산 (Ramamoorthi et al. 공식)
float3 CalculateIrradianceSH(float3 N)
{
    const float C1 = 0.429043;
    const float C2 = 0.511664;
    const float C3 = 0.743125;
    const float C4 = 0.886227;
    const float C5 = 0.247708;

    float3 L00 = g_lblDiffuseSH[0].rgb;
    float3 L1_1 = g_lblDiffuseSH[1].rgb;
    float3 L10 = g_lblDiffuseSH[2].rgb;
    float3 L11 = g_lblDiffuseSH[3].rgb;
    float3 L2_2 = g_lblDiffuseSH[4].rgb;
    float3 L2_1 = g_lblDiffuseSH[5].rgb;
    float3 L20 = g_lblDiffuseSH[6].rgb;
    float3 L21 = g_lblDiffuseSH[7].rgb;
    float3 L22 = g_lblDiffuseSH[8].rgb;

    return (
        C4 * L00 -
        C1 * L22 * (N.x * N.x - N.y * N.y) +
        C3 * L20 * N.z * N.z -
        C5 * L20 +
        2.0 * C1 * L2_2 * N.x * N.y +
        2.0 * C1 * L21 * N.x * N.z +
        2.0 * C1 * L2_1 * N.y * N.z +
        2.0 * C2 * L11 * N.x +
        2.0 * C2 * L1_1 * N.y +
        2.0 * C2 * L10 * N.z
    );
}
```

구면조화 함수를 처리하는 함수 내용

1 - 3. IBL

환경광의 정반사(Specular IBL)를 계산

실시간 연산이 불가능한 환경광 반사 적분 식을 두 부분으로 나누어, 미리 블러 처리된 환경 맵(Prefiltered Map)을 표면의 거칠기(Roughness)에 맞게 샘플링하고, 사전 연산된 2D 텍스처(BRDF LUT)를 활용해 최종적인 간접 Specular를 계산하는 과정입니다.

해당 IBL에 사용된 텍스처는 hdr 4k 텍스처를 Polyheaven에서 구하여 google의 cmgen을 활용하여 SH(Diffuse)와 Specular(mipmap 9단계까지)를 제작하였습니다.

해당 함수에서 roughness의 기본값을 준 이유는 roughness가 0에 한없이 가까워지면 빛이 퍼지지 않고 바늘구멍처럼 한 점에만 극단적으로 모이게 되어 카메라 이동시 마다 번쩍이는 현상이 발생하여서입니다.

이와 같은 수식 예외처리를 통하여 렌더링 안정성을 주었습니다.

```
// Specular IBL 계산
float3 CalculateSpecularIBL(float3 N, float3 V, float3 albedo, float metallic, float roughness)
{
    // 1. 반사 벡터 계산
    float3 R = reflect(-V, N);

    // 2. NdotV 계산
    float NdotV = saturate(dot(N, V));

    // 3. Prefiltered Environment Map LOD 선택
    float maxMipLevel = 8.0;
    float safeRoughness = max(roughness, 0.05);
    float lod = safeRoughness * maxMipLevel;

    // 4. Prefiltered Map 샘플링
    float3 prefilteredColor = g_PrefilteredMap.SampleLevel(g_samLinear, R, lod).rgb;

    // 5. BRDF LUT 샘플링
    float2 brdf = g_BrdfLut.SampleLevel(g_samLinear, float2(NdotV, roughness), 0).rg;

    // 6. F0 계산
    float3 F0 = lerp(float3(0.04, 0.04, 0.04), albedo, metallic);

    // 7. Single Scattering Specular 계산
    float3 specularSingle = F0 * brdf.x + brdf.y;

    // 8. Multiple Scattering Energy Compensation
    // Ess = Single Scattering의 총 에너지
    float Ess = brdf.r + brdf.g;
    Ess = max(Ess, 0.1); // 0으로 나누기 방지

    // 9. 최종 Specular = Single Scattering
    return prefilteredColor * specularSingle;
}
```

언리얼 엔진의 Split-Sum Approximation 방식으로 구현

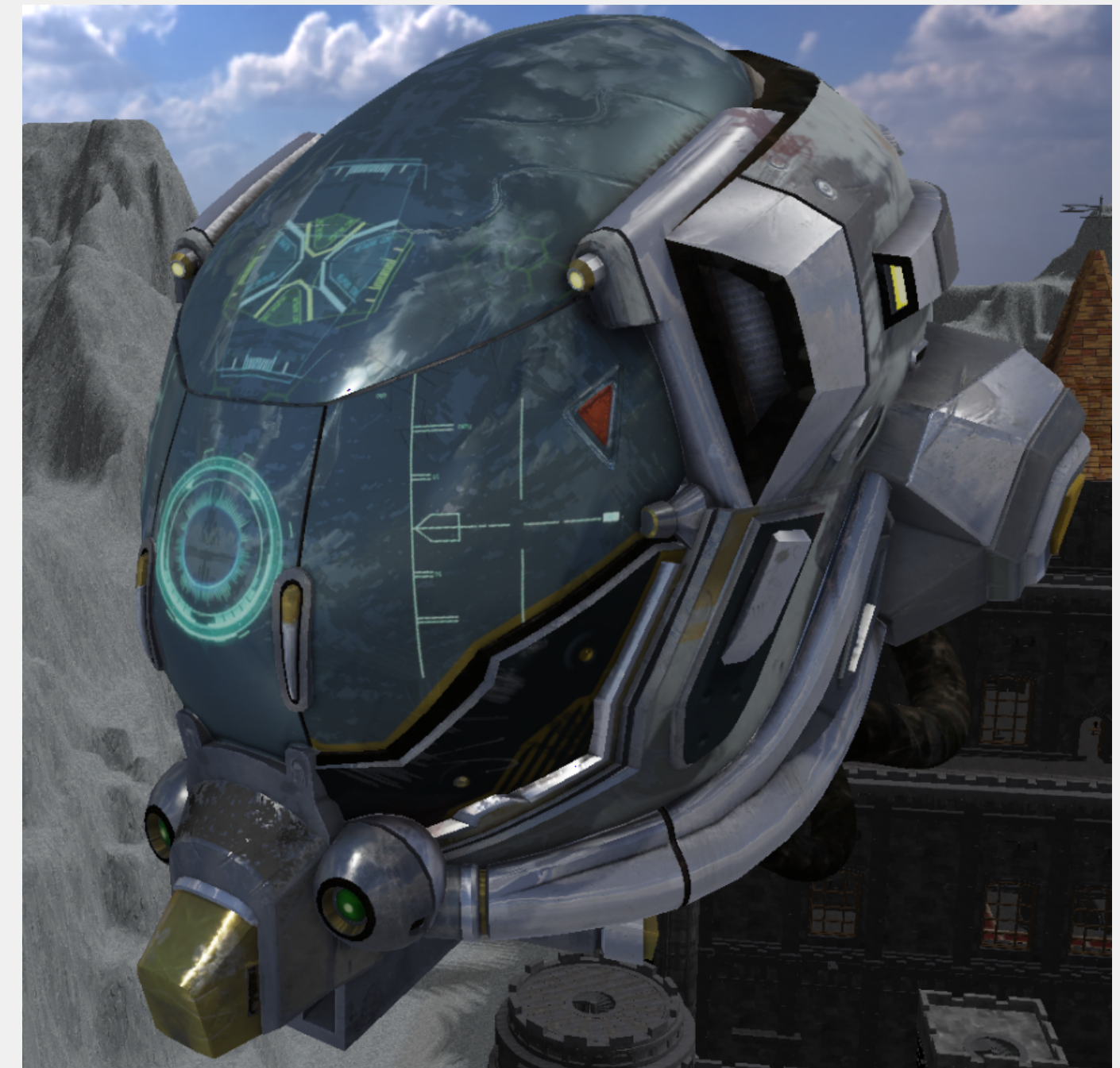
1 - 4. 렌더링 시스템 (PBR & IBL) 적용 결과

자체 D3D12 엔진 렌더링 파이프라인을 통한 텍스처 압축 및 물리 기반 조명 시각화

이전 슬라이드에서 구현한 단일 ORM 텍스처 바인딩 로직을 적용하여 병목 없이 렌더링된 최종 결과물입니다.

Cook-Torrance BRDF 연산을 통해 damaged helmet의 매끄러운 질감(낮은 Roughness)과 물리적으로 알맞게 렌더링했습니다.

헬멧 전면부 및 금속 파츠에 주변 환경(Skybox)이 반사되도록 IBL을 구현하여 사실적인 Ambient Light를 표현했습니다.



자체 엔진에서 gltf 2.0 reference 메쉬 중 하나인 damaged helmet을 자체 엔진에서 렌더링 한 모습

2. 언리얼 블루프린트 제작 게임

[프로젝트명] ASSASSIN VS MUTANT [장르] 3인칭 풀 3D 액션 게임
https://github.com/AACHANJINAA/2025-1_Unreal_LastProject

해당 게임은 플레이어가 암살자가 되어 단검, 대검 등 다양한 무기를 스위칭하며 몰려오는 적(Mutant)과 보스를 처치하는 액션 게임입니다.

단순한 기능 구현을 넘어, 언리얼 엔진 5의 핵심 게임플레이 프레임워크와 다양한 시스템(AI, 애니메이션, UI, 객체 통신 등)을 실무 수준으로 엮어내어 하나의 완성된 게임 사이클을 구축하는 데 집중했습니다.

하단 표와 같이 Blueprint Interface, Event Dispatcher, Behavior Tree 등 엔진이 제공하는 주요 기능들을 구조적으로 활용하여 결합도를 낮추고 확장성 높은 아키텍처를 설계했습니다.

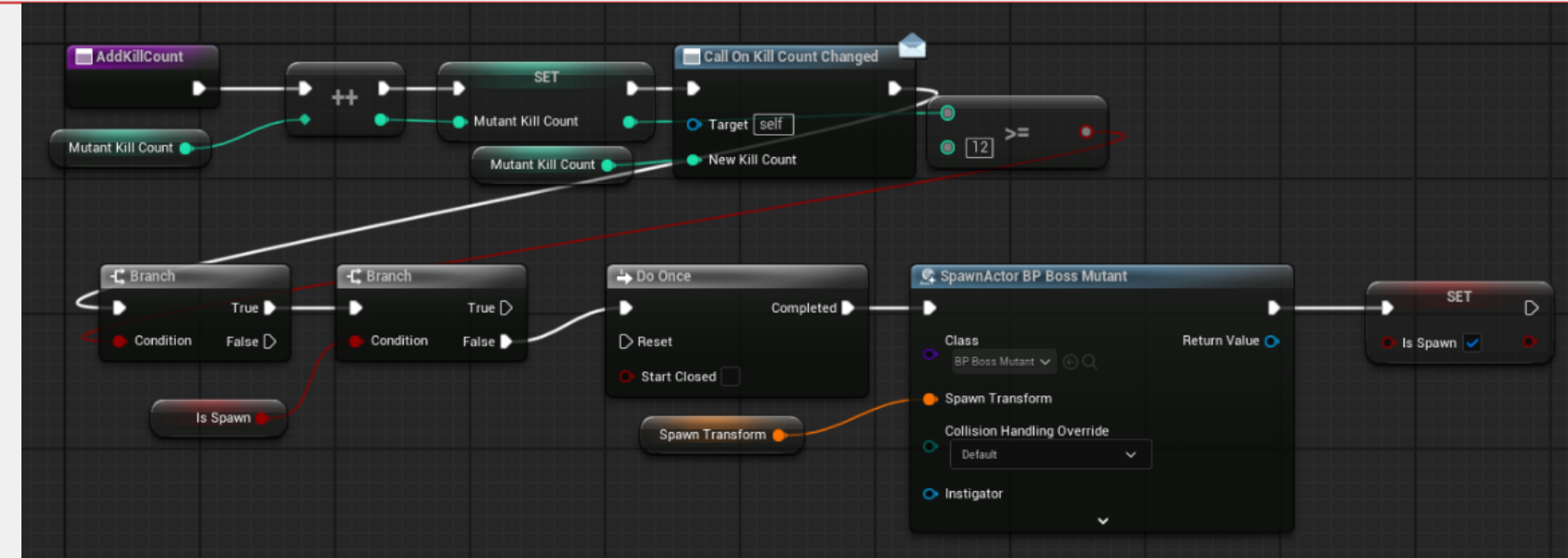
엔진 기능 활용 여부	블루프린트 클래스
	액터 컴포넌트
	컨스트럭션 스크립트
	레벨 블루프린트
	입력 시스템
	게임플레이프레임워크
	애니메이션 몽타주
	머티리얼 인스턴스
	랜드스케이프
	라이팅
	직접 통신
	이벤트 디스패치
	블루프린트 인터페이스
	UMG
	행동 트리



2 - 1. 핵심 기능 설명

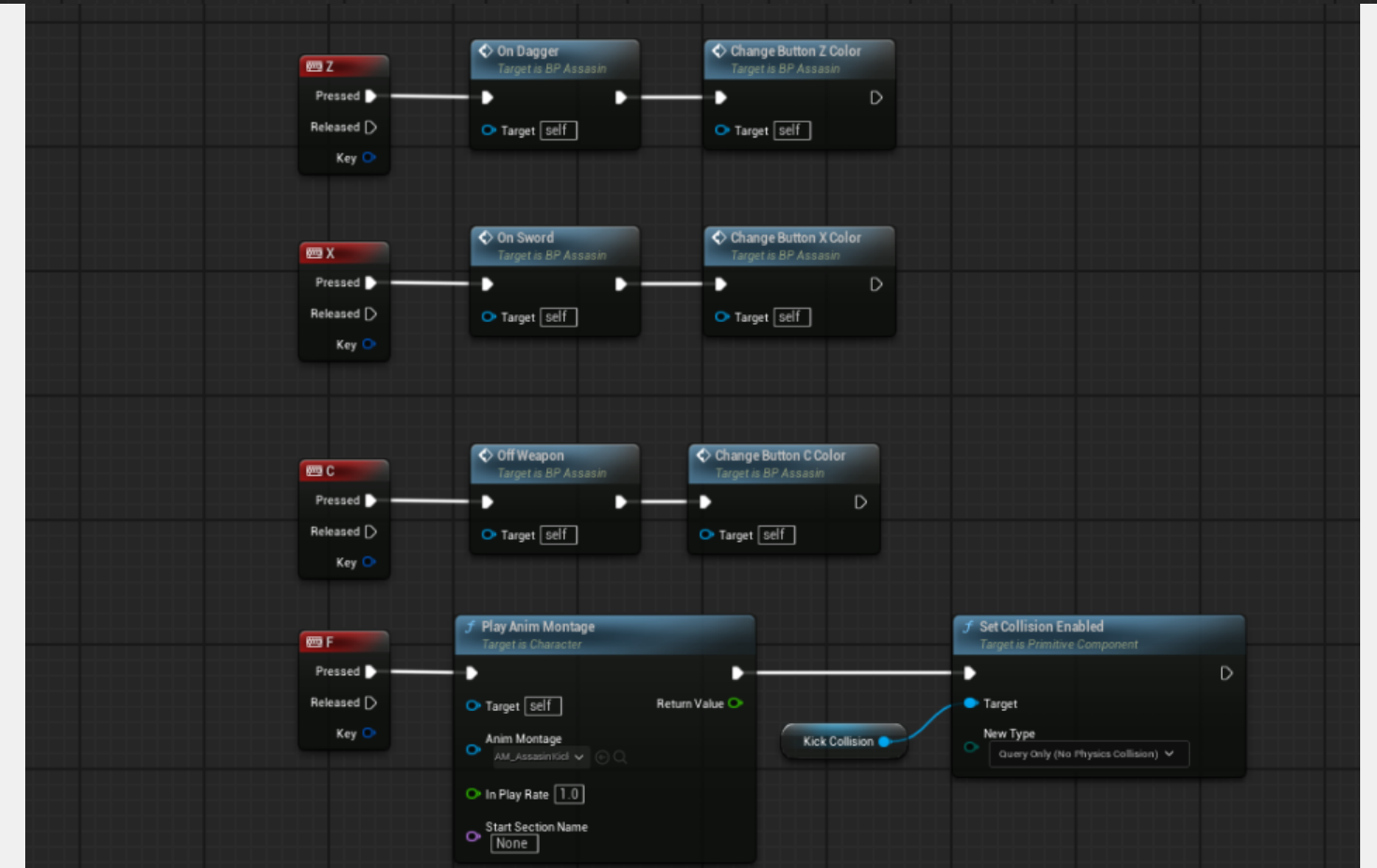
비동기 이벤트 처리 (Event Dispatcher)

BP_AssassinGameMode에서 킬 카운트 등 전역 상태를 중앙 제어하도록 설계했습니다. 상태 변경 시 OnKillCountChanged 이벤트 디스패처를 호출하여, UI가 매 틱(Tick)마다 데이터를 갱신하지 않도록 성능을 최적화했습니다.



다형성 활용 및 의존성 분리 (Blueprint Interface)

플레이어(BP_Assassin)의 상호작용 시스템에 BPI_Interact 인터페이스를 도입했습니다. 오브젝트 픽업 시 하드 캐스팅(Hard Casting)을 방지하고 인터페이스 통신을 사용하여, 새로운 상호작용 객체가 추가되어도 확장하기 쉬운 구조를 구축했습니다.

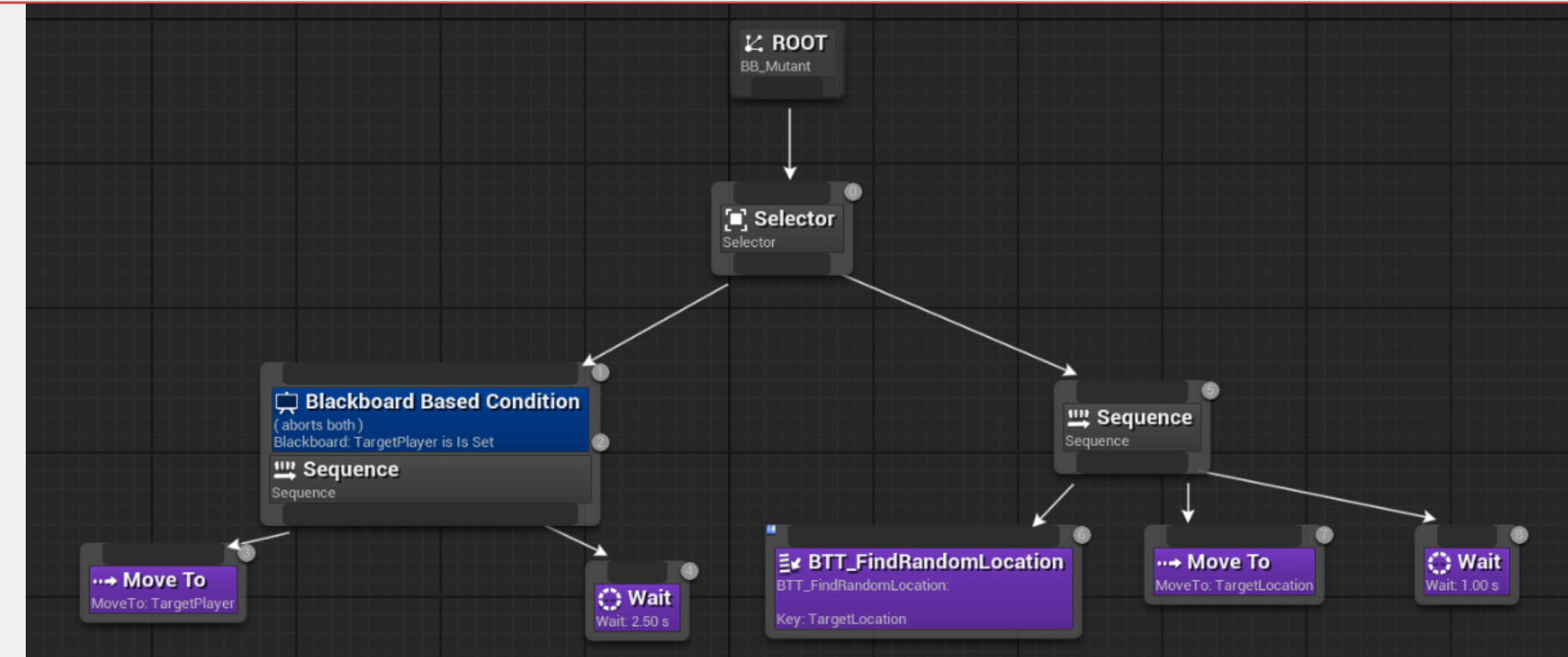


2 - 2. 전투 시스템 및 AI 설계

상태 기반 AI 설계 (Behavior Tree)

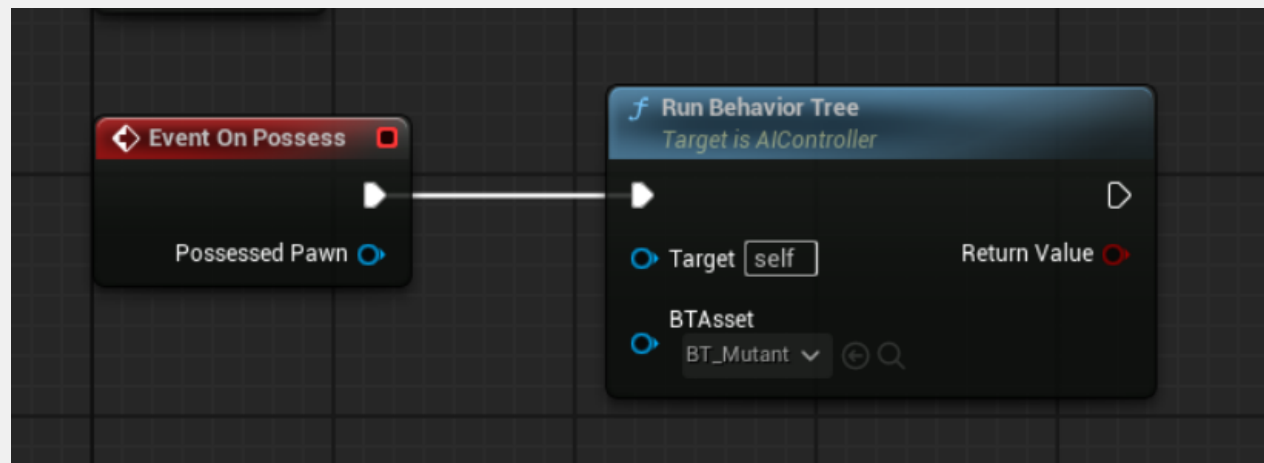
Selector와 Sequence 노드를 활용하여 AI의 행동 우선순위를 정밀하게 설계했습니다.

상황에 따라 [순찰(Patrol) -> 추적(Chase) -> 공격(Attack)] 으로 이어지는 상태 전이 (State Transition) 로직을 Behavior Tree로 시각화 및 모듈화하였습니다.



Custom AI Controller 및 상태 제어

AIC_Mutant 클래스를 직접 구현하여 AI의 기본 동작 상태를 제어하고, Blackboard를 통해 타겟(Player) 인지 상태 및 위치 데이터를 관리했습니다.



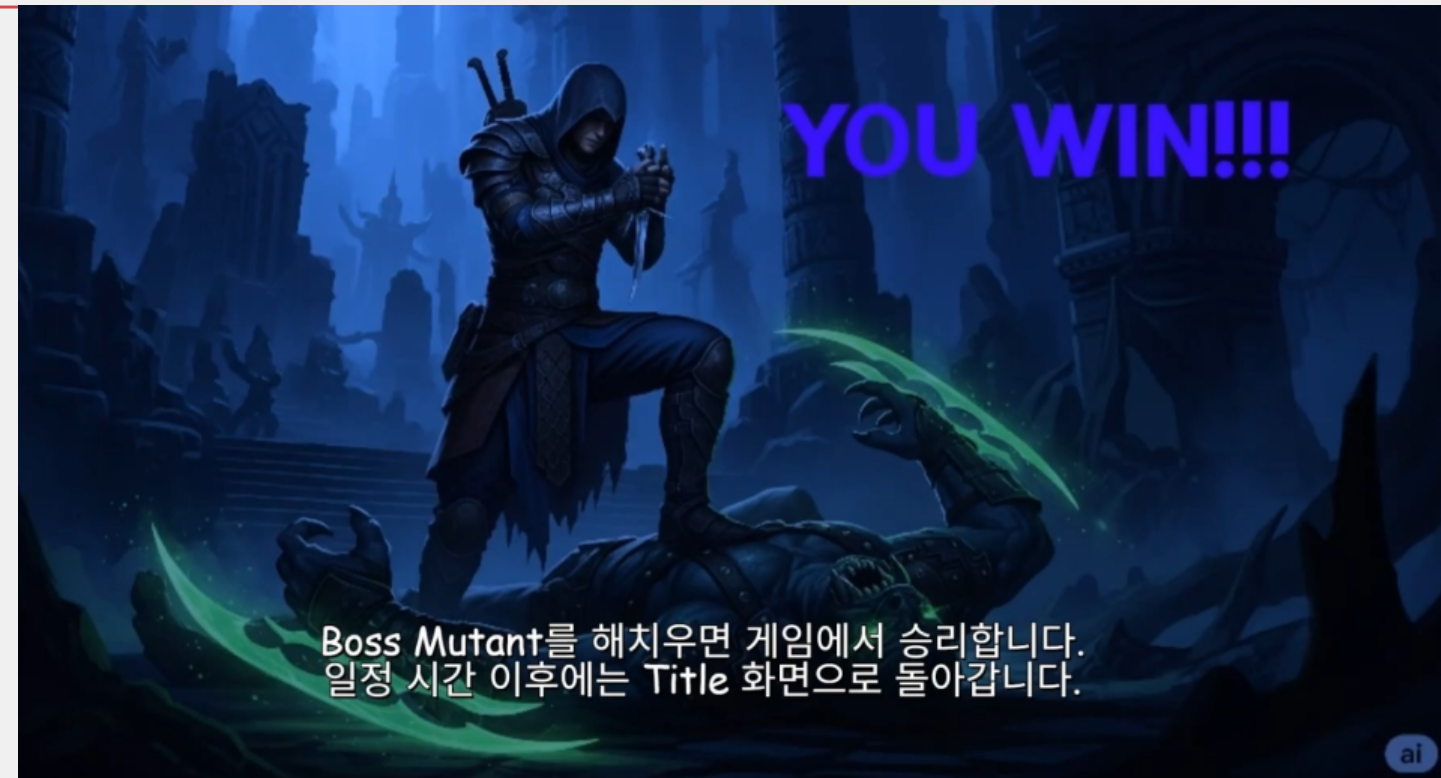
2 - 3. AI를 활용한 리소스 제작

생성형 AI(Generative AI)를 활용한 리소스 파이프라인 최적화 및 개발 효율성 증대

게임의 컨셉(다크 판타지, 암살자 vs 돌연변이)에 부합하는 프롬프트를 Google Gemini에 입력하여, Title, Win, Lose 화면에 사용될 고품질 2D 배경 일러스트를 직접 생성 및 적용했습니다.

프로그래머 1인 개발 또는 소규모 팀 프로젝트 시 가장 큰 병목인 '조건에 맞는 무료 에셋 검색 및 편집' 시간을 획기적으로 단축했습니다.

단순 리소스 수급에 소모되는 시간을 줄인 만큼, 자체 엔진(D3D12)의 렌더링 최적화와 언리얼 엔진의 아키텍처 및 행동 트리(Behavior Tree) 설계 등 클라이언트 프로그래머의 핵심 업무에 더욱 집중할 수 있었습니다.



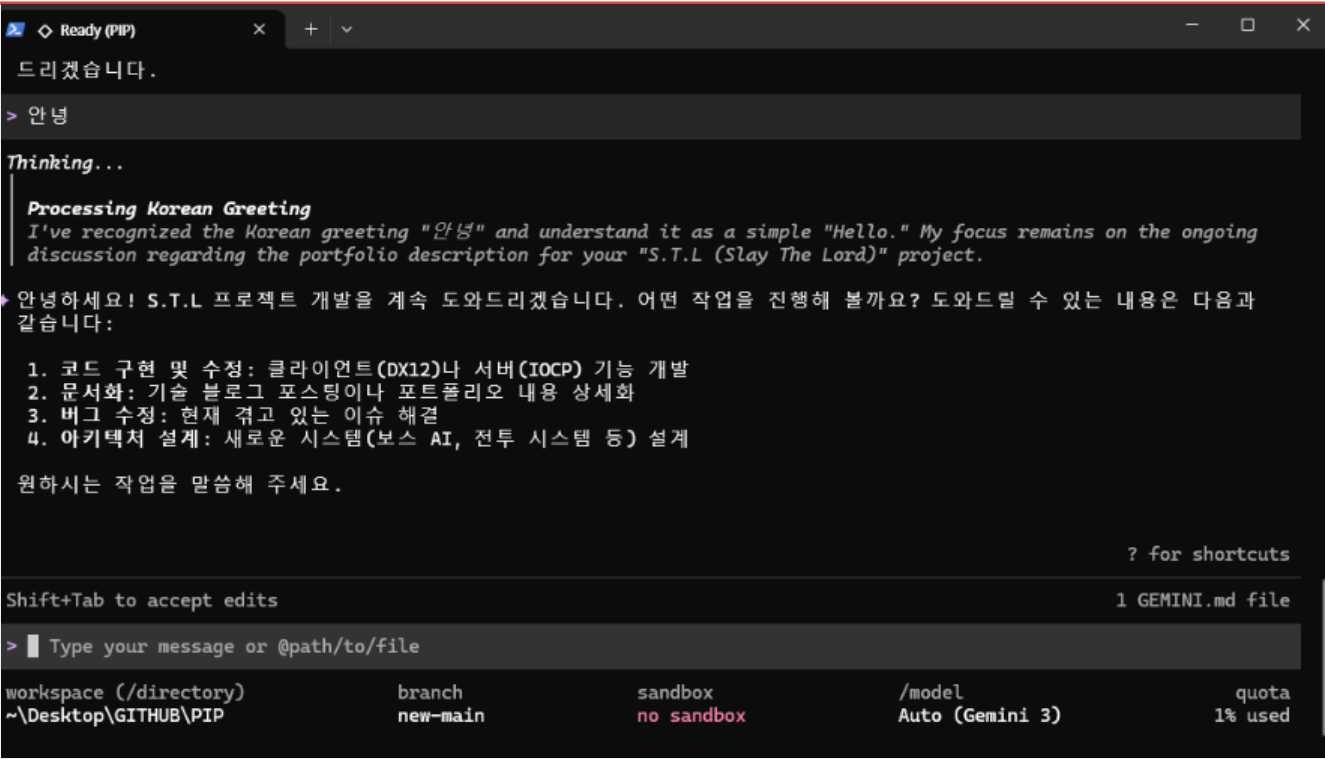
3. AI 활용 능력

개발, 학습에 걸친 AI 활용으로 능률 향상

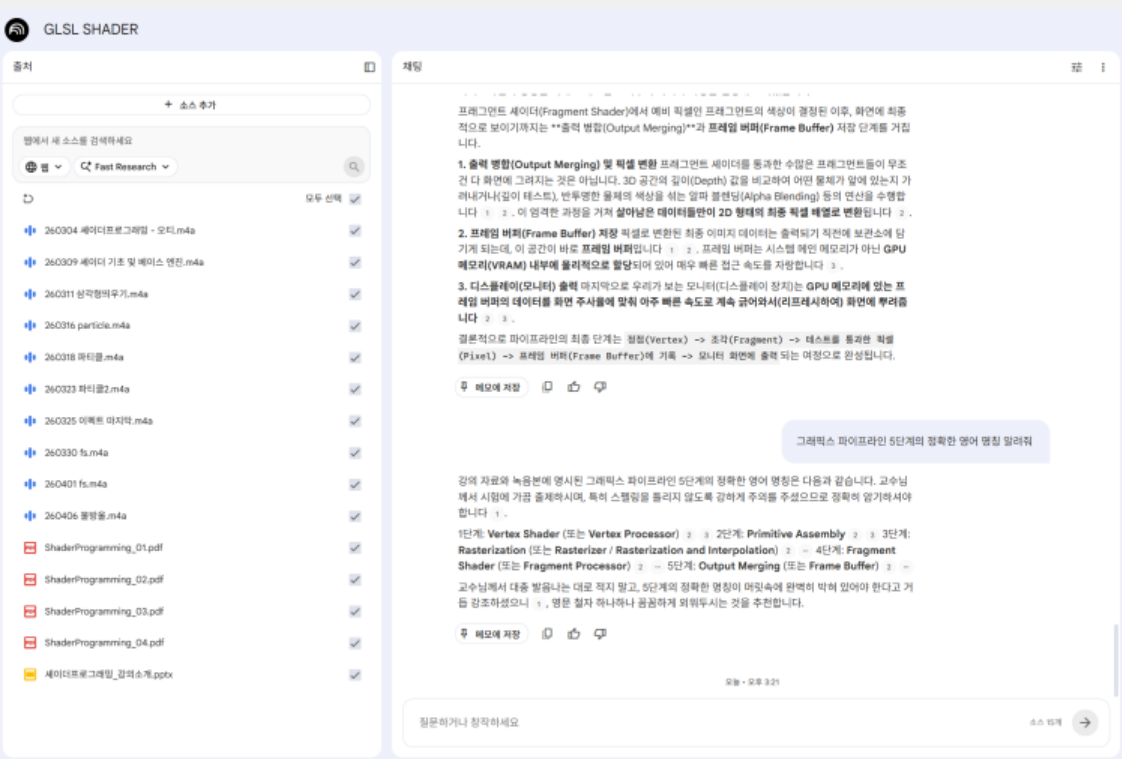
앞에서의 ui를 만든 사례와 같이 저는 개발, 리소스 제작, 공부, 심지어는 일상에서도 AI를 활용합니다. 개발 및 디버깅 시에는 Gemini CLI & GitHub Copilot CLI를 사용하며 해당 폴더의 내용과 제작해놓은 md 파일을 읽어 복잡한 로직을 교차 검증하고 디버깅 효율을 높였습니다.

방대한 엔진 공식 문서나 기술 문서를 Gemini로 빠르게 요약하여 학습 비용을 줄였습니다. 그리고 강의를 녹음 하고 해당 자료들을 넣어 전공 지식 학습하고 필요할 때 즉시 문답 형태로 찾아보는 '나만의 AI 교수님'을 NotebookLM을 통해 만들었습니다.

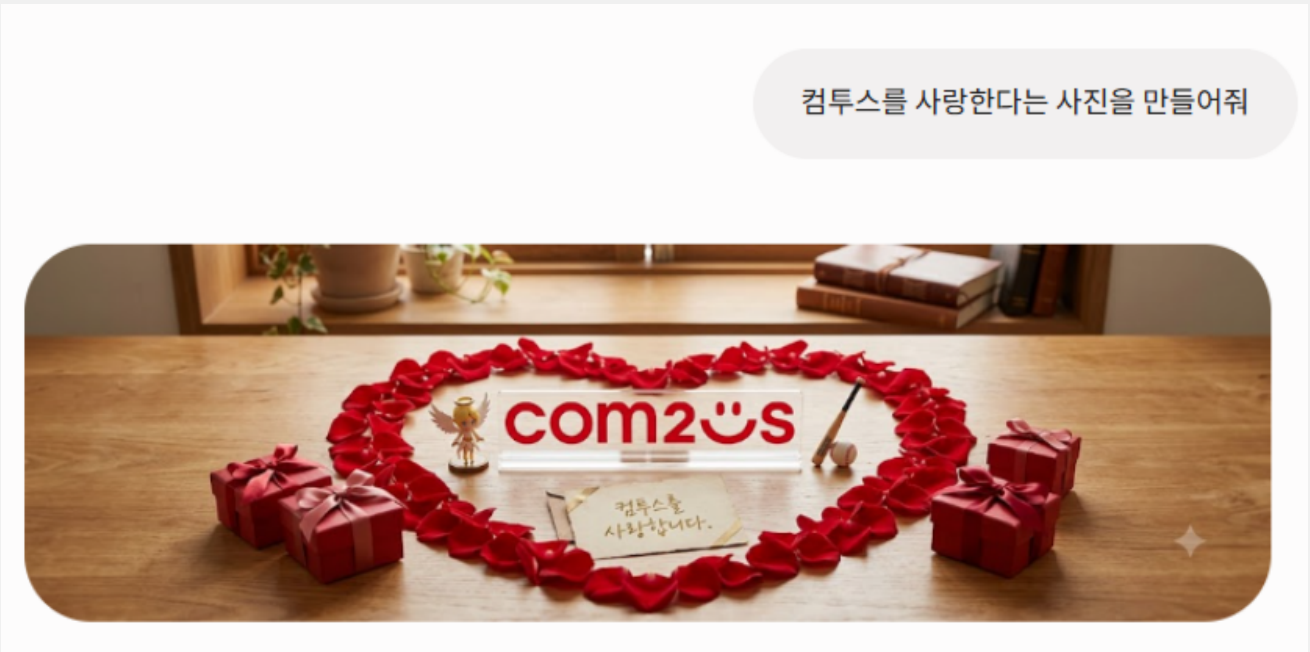
그리고 앞의 사례와 같이 나노바나나 (Generative AI)를 활용하여 2D 리소스 제작을 하여 개발 시간의 큰 병목 중 하나인 리소스 제작 시간을 줄였습니다. 이외에도 Meshy ai와 같은 3D 모델 제작도 몇개 만들어 보았지만, 아직 3D 모델 툴의 리깅의 정확성이라던가, 삼각형의 개수가 너무 많다던가 하는 단점이 있어 게임 개발 시 사용하고 있지는 않습니다.



Gemini CLI

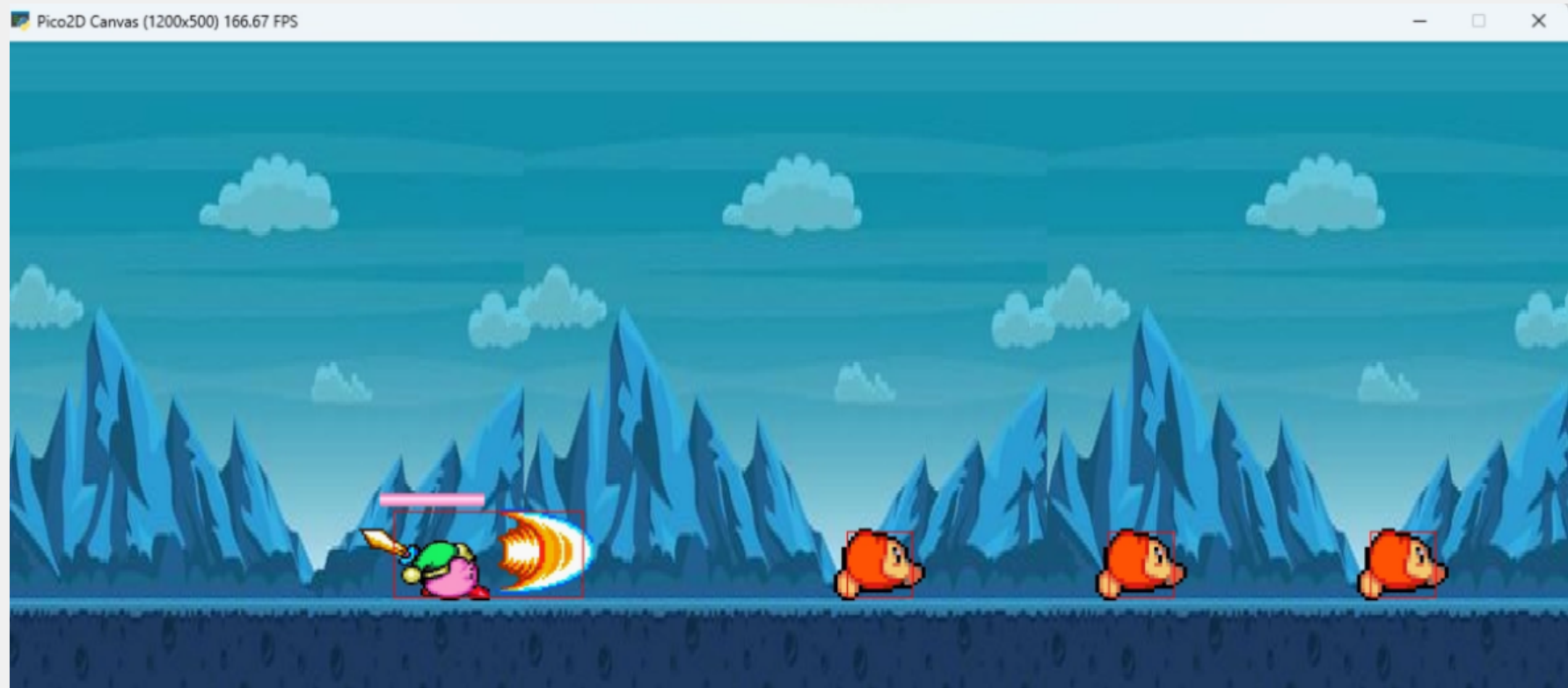


NotebookLM



Gemini (나노바나나)

4. 기타 개발 게임



사용 언어: Python (Pico2D)

- 구현 내용

플레이어와 몬스터의 대기, 이동, 점프 등 다양한 움직임을 유한 상태 기계(FSM) 패턴으로 설계하여 자연스럽게 제어했습니다.

원작의 핵심 재미 요소인 '능력 흡수' 시스템을 구현해 검, 스파크, 폭탄 등 3가지 스킬 모드를 개발했으며, 사진과 같이 모드에 따라 각기 다른 공격 이펙트가 출력되도록 만들었습니다.

게임의 타격감을 높이기 위해 객체들의 충돌 박스(AABB)를 활용하여 스킬 투사체와 몬스터 간의 타격 및 피격 판정을 구현했고, 이 충돌 처리 방식을 응용해 맵 이동 포탈 기능에도 적용했습니다.

-깃허브

<https://github.com/AACHANJINAA/2DGP-final-project-curvy>

연락처

포트폴리오를 검토해 주셔서 감사합니다.

연락처 : 010-4032-9600

이메일: ckswl0916@gmail.com

GitHub: <https://github.com/AACHANJINAA>

플레이어들에게 행복한 시간을 가질 수 있는 경험을 제공하고 싶습니다.
끊임없이 학습하고 성장하며, 팀과 함께 게임 기술을 만들어가는 개발자가 되겠습니다.

추가 질문이나 프로젝트 상세 내용이 궁금하시면 언제든지 연락 주시기 바랍니다! 감사합니다!